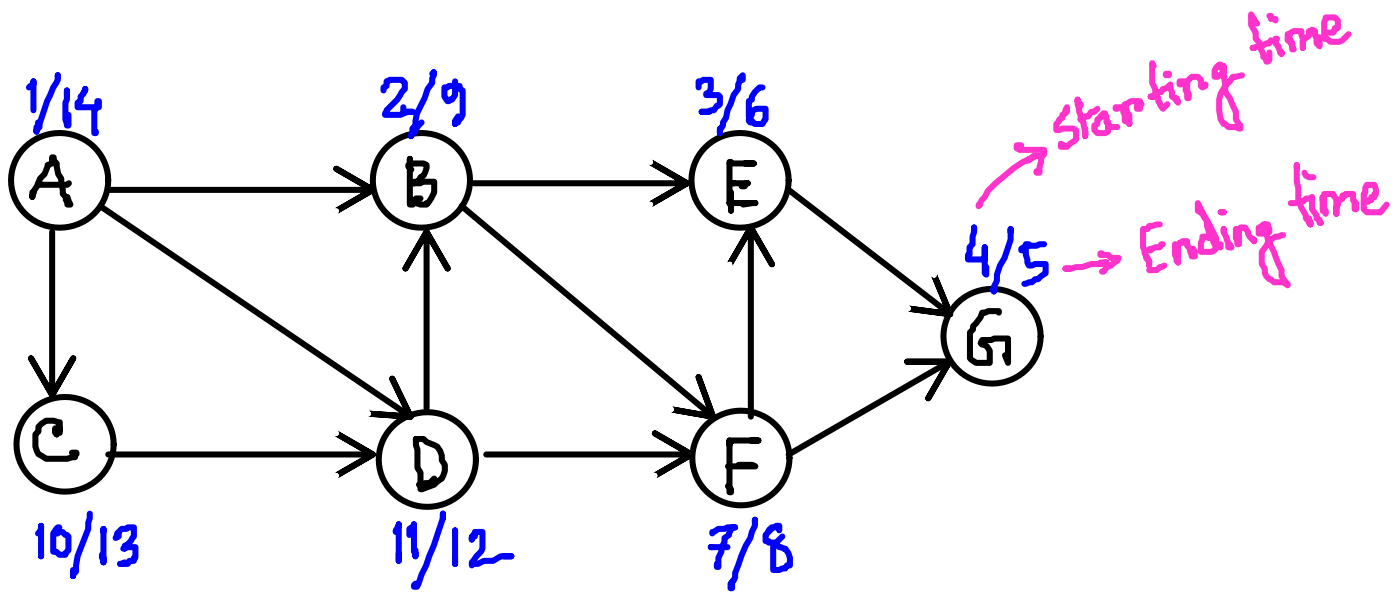
Depth First Search (DFS)



Time = 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

(u) E — 3/10

(v) G — 5/6

E → G

Pre → Starting [

Post → Ending]

Call :

DFS(A) → C

DFS(B) → F

DFS(E)

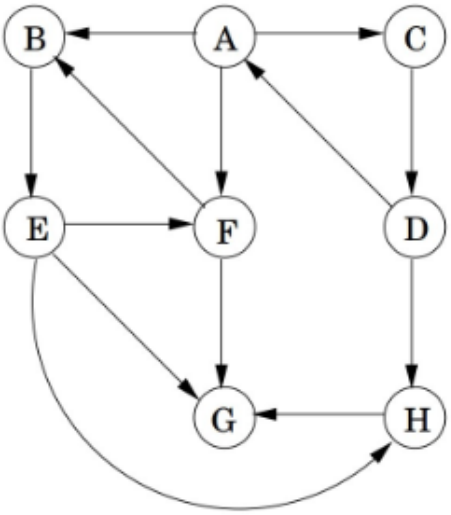
DFS(G)

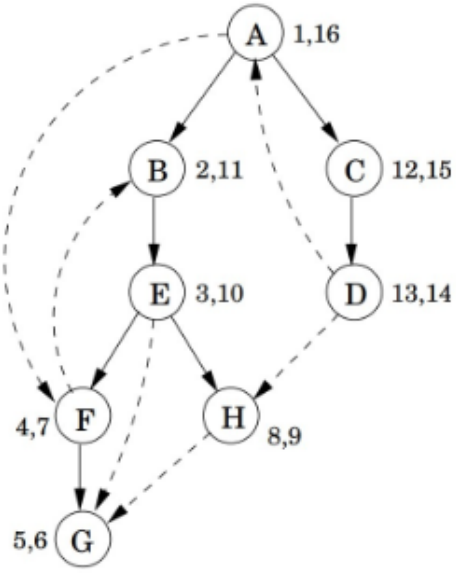
3 < 5 < 6 < 10

Pre[< Pre[< Post] < Post]

u v v u

→ Forward Edge





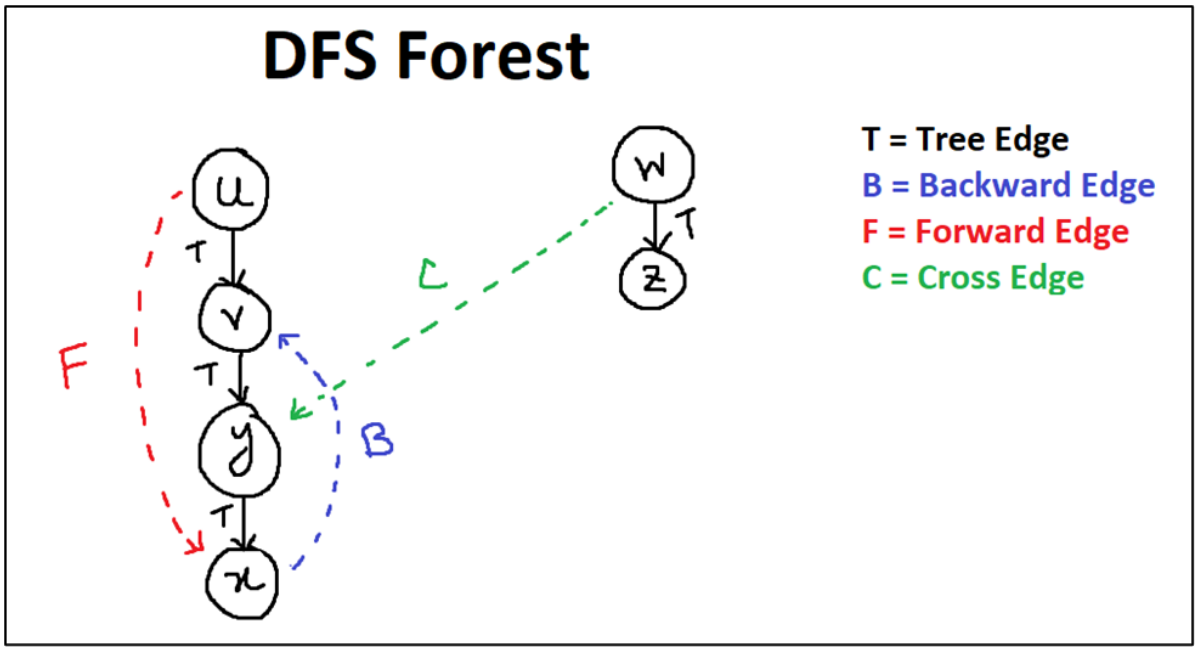
Edge Type (u, v)	**Pre/Post-Order**
Tree/forward	$\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$
Back	$\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$
Cross	$\text{pre}(v) < \text{post}(v) < \text{pre}(u) < \text{post}(u)$

`pre/post` *ordering for* (u, v)	*Edge type*
$[_u \; [_v \;]_v \;]_u$	Tree/forward
$[_v \; [_u \;]_u \;]_v$	Back
$[_v \;]_v \; [_u \;]_u$	Cross

DFS Applications:

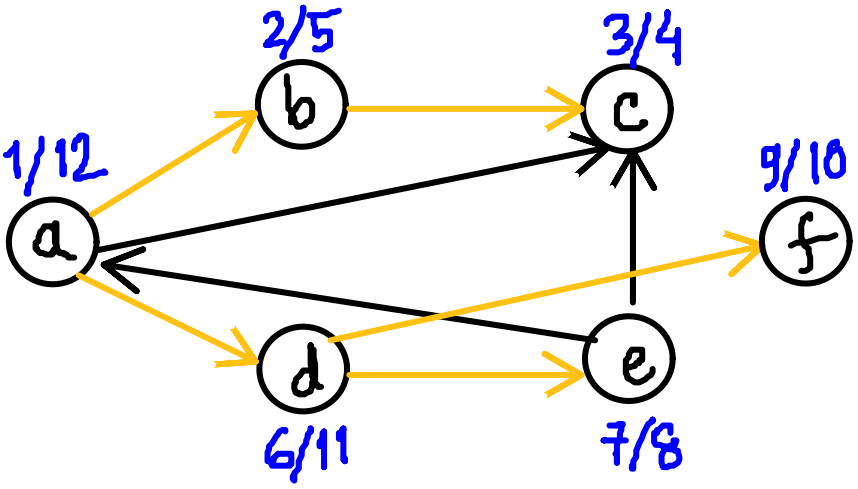
After performing DFS, there are 4 types of edges:

1) Tree Edge - edge generated during DFS
2) Back Edge - edge from a node to one of its ancestors in a DFS tree
3) Forward Edge - edge from a node to one of its descendants in a DFS tree
4) Cross Edge - edge connecting nodes from different branches



For each edge (u -> v) in the graph:

Tree/Foward: u.start < v.start < v.finish < u.finish
Back: v.start < u.start < u.finish < v.finish
Cross: v.start < v.finish < u.start < u.finish



Tree : $a \to b \to c$, $a \to d \to e$, $a \to d \to f$

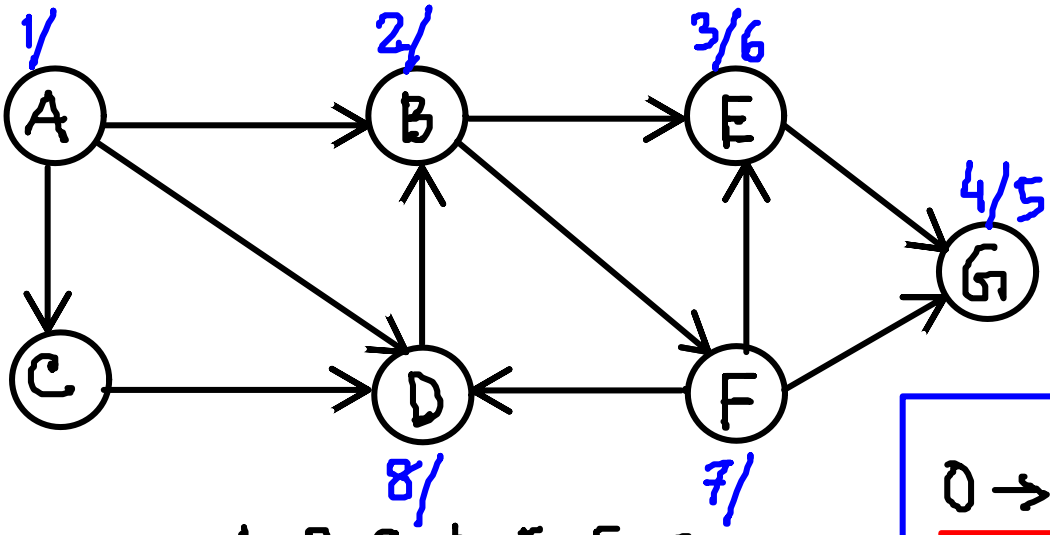
Forward : $a \to c$

Back : $a \to e$

Cross : $\left.\begin{matrix} a \to b \to e \\ a \to d \to e \end{matrix}\right\} a \to e$

Simulation

Directed :



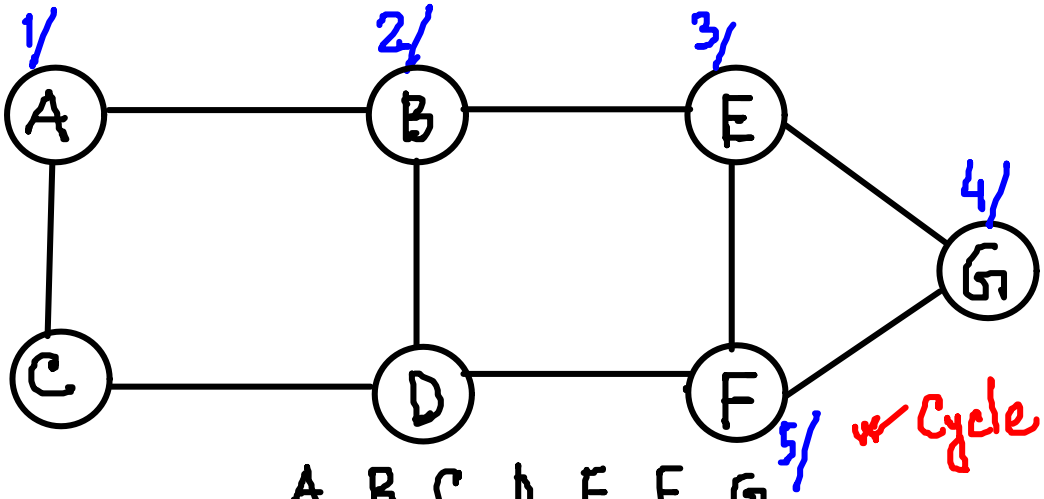
	A	B	C	D	E	F	G
Color =	∅	∅	0	∅	∅	∅	∅
	1	1		1	✗ 2	1	✗ 2

0 → Visited

1 → Processing ← Cycle

2 → Processing ← Done

Undirected :



	A	B	C	D	E	F	G
Visited =	∅	∅	0	0	∅	∅	∅
	1	1			1	1	1

Adjacent Node is already visited but not my Parent --> Cycle

	A	B	C	D	E	F	G
Parent =	-1	A			B	G	E

Depth-First Search: The Code

```
DFS(G)
{
   for each vertex u ∈ G->V
   {
      u->color = WHITE;
   }
   time = 0;
   for each vertex u ∈ G->V
   {
      if (u->color == WHITE)
         DFS_Visit(u);
   }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   time = time+1;
   u->d = time;
   for each v ∈ u->Adj[]
   {
      if (v->color == WHITE)
         DFS_Visit(v);
   }
   u->color = BLACK;
   time = time+1;
   u->f = time;
}
```

Running time / Time Compexity of DFS = **O(V+E)**